

Resumen ampliado — Capítulo 1

Introducción al diseño e implementación de compiladores

*Basado en el Capítulo 1 de Aho, Lam, Sethi y Ullman, **Compiladores: principios, técnicas y herramientas** (“Libro del Dragón”)*

Material de repaso — Cátedra de Compiladores

Índice

1	Introducción	2
2	Procesadores de lenguaje	2
2.1	Compiladores vs. intérpretes	2
2.2	El sistema completo de procesamiento de lenguaje	3
3	La estructura interna de un compilador	3
3.1	Análisis léxico	5
3.2	Lexemas, tokens y patrones	5
3.3	Análisis sintáctico	6
3.4	Especificación sintáctica	7
3.5	Árbol de sintaxis abstracta	8
3.6	Análisis semántico	9
3.7	Tabla de símbolos	9
3.8	Manejo de errores	10
3.9	Generación de código intermedio	10
3.10	Optimización de código	11
3.11	Generación de código destino	11
3.12	Front-end y back-end	11
3.13	Fases y pasadas	12
3.14	Herramientas para la construcción de compiladores	13
3.15	Proceso completo	13
4	Ejemplo completo: recorrido de una instrucción a través de todas las fases	14
5	Uso de C como lenguaje destino	15
6	De la teoría del Capítulo 1 a la práctica: bases para Flex y Bison	16
6.1	Expresiones regulares como patrones léxicos: la base de Flex	16
6.2	Gramáticas libres de contexto y árboles sintácticos: la base de Bison	16
6.3	El flujo Flex + Bison y su correspondencia con las fases del compilador	17
7	Panorama complementario	17
7.1	La evolución de los lenguajes de programación	17
7.2	La ciencia de construir un compilador	18
7.3	Aplicaciones de la tecnología de compiladores	18
8	Conclusión	18

1. Introducción

Los lenguajes de programación son la forma en que las personas describen cálculos a las máquinas. Sin embargo, ningún programa fuente puede ejecutarse directamente: antes debe **traducirse** a un formato que la computadora pueda procesar. Los sistemas de software que realizan esta traducción se llaman **compiladores**.

El estudio de los compiladores no es solamente el estudio de una herramienta puntual: las ideas y técnicas que se utilizan para construirlos (autómatas finitos, expresiones regulares, gramáticas libres de contexto, análisis de flujo de datos, optimización) reaparecen en muchas otras áreas de las ciencias de la computación. Por esta razón, comprender la estructura general de un compilador es la base indispensable para todo lo que veremos después en la materia, en particular para las herramientas **Flex** (generador de analizadores léxicos) y **Bison** (generador de analizadores sintácticos), que automatizan justamente las dos primeras fases que se describen en este capítulo.

2. Procesadores de lenguaje

En su forma más simple, un **compilador** es un programa que lee un programa escrito en un **lenguaje fuente** y lo traduce a un programa equivalente escrito en un **lenguaje destino**. Una función esencial del compilador es, además, reportar los errores del programa fuente que detecte durante la traducción.

$$\text{programa fuente} \longrightarrow \boxed{\text{Compilador}} \longrightarrow \text{programa destino}$$

Si el programa destino es un programa ejecutable en lenguaje máquina, el usuario puede ejecutarlo directamente para procesar entradas y producir salidas:

$$\text{entrada} \longrightarrow \boxed{\text{Programa destino}} \longrightarrow \text{salida}$$

2.1. Compiladores vs. intérpretes

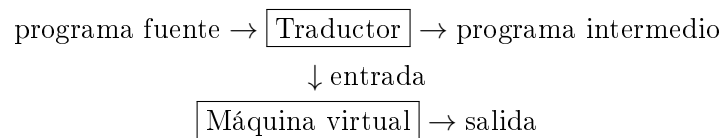
Un **intérprete** es otro tipo de procesador de lenguaje muy común. En vez de producir un programa destino como traducción, el intérprete da la apariencia de ejecutar directamente las operaciones especificadas en el programa fuente, usando las entradas suministradas por el usuario:

$$\text{programa fuente, entrada} \longrightarrow \boxed{\text{Intérprete}} \longrightarrow \text{salida}$$

En términos de eficiencia y diagnóstico de errores existe un compromiso:

- El programa destino en lenguaje máquina que produce un **compilador** suele ser **más rápido** que un intérprete al mapear entradas a salidas, porque la traducción se realiza una sola vez.
- Un **intérprete** suele ofrecer **mejores diagnósticos de error**, ya que ejecuta el programa fuente instrucción por instrucción y puede reportar el contexto exacto en el que ocurre un fallo.

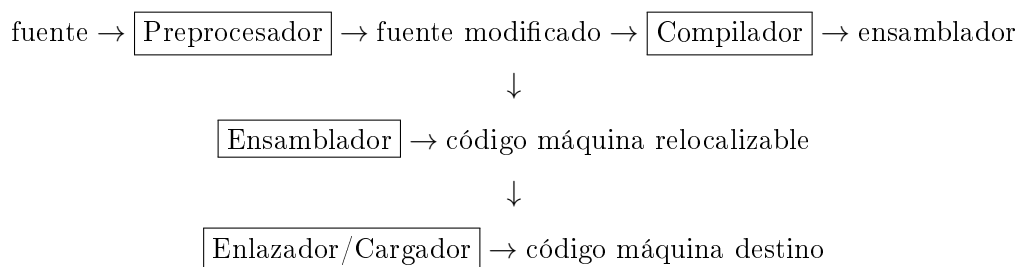
Procesadores híbridos. Muchos lenguajes modernos combinan ambas estrategias. El caso clásico es Java: el programa fuente se compila primero a un formato intermedio llamado *bytecode*, y luego una máquina virtual interpreta esos bytecodes. Esto permite que el mismo bytecode compilado en una máquina se ejecute en otra distinta (incluso a través de una red). Para acelerar aún más la ejecución, los compiladores *just-in-time* (JIT) traducen los bytecodes a lenguaje máquina justo antes de ejecutarlos.



2.2. El sistema completo de procesamiento de lenguaje

Para producir un ejecutable final, generalmente se requieren varios programas además del compilador propiamente dicho:

1. Un **preprocesador** recolecta el programa fuente (que puede estar dividido en varios módulos/archivos) y expande *macros*, es decir, fragmentos de código abreviados de uso frecuente, en instrucciones del lenguaje fuente.
2. El **compilador** traduce el programa fuente (ya preprocesado) a un programa destino, que muchas veces es código en **lenguaje ensamblador**, porque resulta más fácil de generar y de depurar que el código máquina directo.
3. Un **ensamblador** procesa el código ensamblador y produce **código máquina relocable**.
4. Un **enlazador (linker)** resuelve las referencias a direcciones de memoria externas, es decir, cuando código de un archivo hace referencia a algo definido en otro archivo, y combina el código relocable con archivos objeto y bibliotecas.
5. Un **cargador (loader)** reúne todos los archivos objeto ejecutables en memoria para su ejecución.



Esta separación en programas independientes es una primera manifestación de una idea que reaparecerá constantemente en la materia: dividir un problema grande y complejo (traducir un lenguaje) en **etapas más pequeñas y manejables**, cada una con una responsabilidad claramente delimitada.

3. La estructura interna de un compilador

Si “abrimos la caja” del compilador, encontramos que la traducción se realiza en dos grandes partes:

- El **análisis** (*front-end*) descompone el programa fuente en sus componentes, les impone una estructura gramatical y construye a partir de ellos una **representación intermedia**. Si detecta que el programa está mal formado sintácticamente o que su semántica es inconsistente, debe emitir mensajes de error útiles. El análisis también recolecta información sobre el programa en la **tabla de símbolos**.

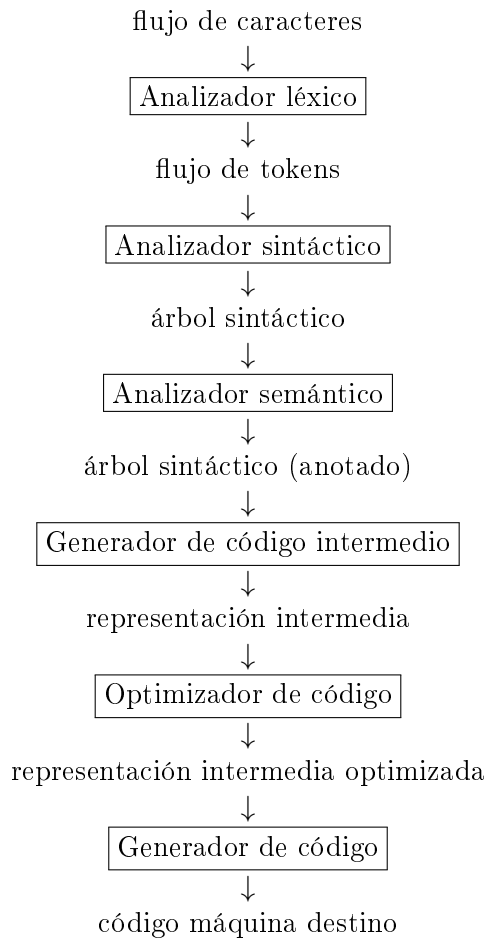
- La **síntesis** (*back-end*) construye el programa destino a partir de la representación intermedia y de la información almacenada en la tabla de símbolos.

En nuestro caso, el lenguaje destino de todo el proceso será **C**, de modo que el compilador que construiremos terminará generando un programa en C, el cual luego será compilado por un compilador de C ya existente.

De forma general:

Programa fuente → Análisis → Representación intermedia → Optimización → Código destino

Analizado con más detalle, el proceso de compilación opera como una **secuencia de fases**, cada una de las cuales transforma una representación del programa fuente en otra:



La **tabla de símbolos** se utiliza de manera transversal en todas las fases, ya que almacena información sobre todo el programa fuente. La fase de **optimización de código independiente de la máquina**, entre el front-end y el back-end, es **opcional**: su objetivo es transformar la representación intermedia para que el back-end pueda producir mejor código destino.

En la práctica, varias fases pueden agruparse y las representaciones intermedias entre fases agrupadas no necesitan construirse explícitamente; volveremos sobre esta idea en la sección 3.13 (*Fases y pasadas*).

Como ejemplo guía utilizaremos el siguiente programa fuente, igual que en el resto de la cátedra:

```
int x = 10 + 20;
```

3.1. Análisis léxico

El análisis léxico, también llamado **escaneo**, constituye la primera fase del compilador. El analizador léxico lee el flujo de caracteres del programa fuente y los agrupa en secuencias significativas llamadas **lexemas**. Para cada lexema, produce como salida un **token** de la forma:

⟨nombre-token, valor-atributo⟩

que se pasa a la fase siguiente, el análisis sintáctico. El **nombre-token** es un símbolo abstracto utilizado durante el análisis sintáctico (por ejemplo, **id**, **+**), y el **valor-atributo** suele apuntar a una entrada en la **tabla de símbolos** con información adicional sobre ese token (por ejemplo, el nombre y el tipo de un identificador).

De esta manera, el análisis léxico separa la **representación concreta** escrita por el programador de la **categoría** que interesa a las fases posteriores, y el analizador léxico **ignora los espacios en blanco** que separan a los lexemas.

Por lo tanto, el análisis léxico responde principalmente a la pregunta:

¿Qué unidades léxicas aparecen en la secuencia de caracteres del programa fuente?

El analizador léxico **no** determina si esas unidades pueden combinarse correctamente: esa responsabilidad corresponde al analizador sintáctico (sección 3.3).

Para nuestro ejemplo:

```
int x = 10 + 20;
```

podemos identificar los siguientes lexemas:

```
int
x
=
10
+
20
;
```

Cada uno de estos lexemas pertenece a una categoría léxica (token):

```
INT
IDENTIFIER
ASSIGN
INTEGER
PLUS
INTEGER
SEMICOLON
```

Relevancia para Flex. Esta fase es exactamente la que automatiza **Flex**: a partir de una especificación de patrones (expresiones regulares) para cada categoría léxica, Flex genera automáticamente el código en C de una función (típicamente `yylex()`) capaz de leer el flujo de caracteres, agruparlo en lexemas y devolver, uno por uno, los tokens correspondientes. Retomaremos esta conexión en la sección 6.

3.2. Lexemas, tokens y patrones

Es fundamental distinguir con precisión estos tres conceptos, ya que son la base conceptual de cualquier especificación léxica (y, por lo tanto, de cualquier archivo `.l` de Flex):

- Un **lexema** es una secuencia concreta de caracteres del programa fuente que forma una unidad, por ejemplo `contador`, `25` o `=`.
- Un **token** es la categoría léxica que se asigna a un lexema, representada como un par $\langle \text{nombre-token}, \text{valor-atributo} \rangle$. El **valor-atributo** sólo es necesario cuando existen varios lexemas posibles para el mismo token (por ejemplo, distintos identificadores); tokens como `=` no lo necesitan.
- Un **patrón** es la **regla** —típicamente una expresión regular— que describe qué secuencias de caracteres pueden formar los lexemas de un determinado token.

Para nuestro lenguaje de ejemplo, la especificación léxica inicial es:

Token	Patrón (expresión regular)	Ejemplo de lexema
INT	<code>int</code>	<code>int</code>
IDENTIFIER	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	<code>x</code> , <code>contador</code>
INTEGER	<code>[0-9]+</code>	<code>10</code> , <code>250</code>
ASSIGN	<code>=</code>	<code>=</code>
PLUS	<code>+</code>	<code>+</code>
SEMICOLON	<code>;</code>	<code>;</code>

Cuadro 1: Especificación léxica inicial del lenguaje

Nótese que la columna “patrón” de la tabla 1 **es, literalmente**, lo que se escribiría en la sección de definiciones de tokens de un archivo de especificación de Flex: cada regla de Flex es, en esencia, un par (*patrón* en forma de expresión regular, *acción* en C que construye y devuelve el token correspondiente).

Para el programa completo:

```
int x = 10 + 20;
```

el analizador léxico produce la secuencia de tokens:

```
INT
IDENTIFIER(x)
ASSIGN
INTEGER(10)
PLUS
INTEGER(20)
SEMICOLON
```

Los tokens que poseen un valor asociado (como los identificadores y los literales enteros) transportan esa información adicional, por ejemplo `IDENTIFIER(x)` o `INTEGER(10)`. El identificador también puede representarse mediante una referencia a una entrada de la tabla de símbolos, siguiendo la notación clásica del Libro del Dragón:

(**id**, 1)

donde **id** representa la categoría del token (identificador) y 1 identifica la entrada correspondiente en la tabla de símbolos (sección 3.7).

3.3. Análisis sintáctico

El análisis sintáctico, o *parsing*, es la segunda fase del compilador. El analizador sintáctico (*parser*) recibe la secuencia de tokens producida por el analizador léxico y **determina si esa secuencia puede derivarse** de acuerdo con la gramática del lenguaje, construyendo además una

representación intermedia en forma de árbol —típicamente un **árbol sintáctico**— que describe la estructura gramatical del flujo de tokens.

Mientras que el análisis léxico determina **qué** unidades aparecen en el programa, el análisis sintáctico determina **cómo** pueden combinarse esas unidades. Por lo tanto, responde a la pregunta:

¿Cómo se combinan los tokens para formar construcciones válidas del lenguaje?

Por ejemplo, la secuencia:

INT IDENTIFIER ASSIGN INTEGER PLUS INTEGER SEMICOLON
--

corresponde a una declaración válida (`int x = 10 + 20;`), mientras que:

INT IDENTIFIER ASSIGN PLUS INTEGER SEMICOLON
--

contiene tokens individualmente válidos, pero cuya organización **no** corresponde a ninguna construcción permitida por la gramática: es un **error sintáctico**.

En un árbol sintáctico, cada **nodo interior** representa una operación, y los **hijos** de ese nodo representan sus argumentos. Esta representación en árbol es la que permite, por ejemplo, expresar de forma no ambigua la **precedencia de operadores**: para `10 + 20 * 3`, el árbol debe reflejar que la multiplicación se agrupa primero, de modo que la expresión se interprete como $10 + (20 \times 3)$ y no como $(10 + 20) \times 3$.

Relevancia para Bison. Esta es exactamente la fase que automatiza **Bison**: a partir de una **gramática libre de contexto** (una colección de reglas de producción como las de la sección 3.4), Bison genera un analizador LALR capaz de reconocer si una secuencia de tokens es derivable desde el símbolo inicial de la gramática, y de ejecutar **acciones semánticas** asociadas a cada regla —típicamente, la construcción de nodos de un árbol de sintaxis abstracta—.

3.4. Especificación sintáctica

La sintaxis de un lenguaje de programación se especifica formalmente mediante una **gramática libre de contexto** (GLC). Una gramática de este tipo consiste en un conjunto de **símbolos no terminales** (categorías sintácticas, como $\langle \textit{expresion} \rangle$), **símbolos terminales** (los tokens producidos por el analizador léxico) y **producciones** que indican cómo un no terminal puede reescribirse en una secuencia de terminales y no terminales.

Para el lenguaje propuesto, una gramática inicial es:

$$\begin{aligned} \langle \textit{programa} \rangle &\rightarrow \langle \textit{declaracion} \rangle \\ \langle \textit{declaracion} \rangle &\rightarrow \langle \textit{tipo} \rangle \text{ IDENTIFIER ASSIGN } \langle \textit{expresion} \rangle \text{ SEMICOLON} \\ \langle \textit{tipo} \rangle &\rightarrow \text{INT} \\ \langle \textit{expresion} \rangle &\rightarrow \langle \textit{expresion} \rangle \text{ PLUS } \langle \textit{termino} \rangle \\ \langle \textit{expresion} \rangle &\rightarrow \langle \textit{termino} \rangle \\ \langle \textit{termino} \rangle &\rightarrow \text{INTEGER} \end{aligned}$$

Esta gramática establece qué secuencias de tokens forman declaraciones y expresiones válidas. Nótese que la producción recursiva de $\langle \textit{expresion} \rangle$ (que se refiere a sí misma del lado izquierdo, con **PLUS** como operador entre una expresión y un término) es lo que permite generar expresiones con una cantidad arbitraria de sumandos, y su forma **recursiva a izquierda** determina que **+** se agrupe de izquierda a derecha; el tratamiento riguroso de este tipo de ambigüedades y de la

precedencia de operadores mediante gramáticas es, precisamente, uno de los temas centrales que veremos en la clase de gramáticas.

Para el programa `int x = 10 + 20;`, el analizador sintáctico puede construir un árbol de derivación equivalente a:

```
declaracion
|-- tipo -> INT
|-- IDENTIFIER -> x
|-- ASSIGN -> =
|-- expresion
|   |-- expresion
|   |   '-- termino
|   |       '-- INTEGER -> 10
|   |-- PLUS
|   '-- termino
|       '-- INTEGER -> 20
'-- SEMICOLON
```

Esta estructura constituye una representación de la organización sintáctica del programa; es lo que en Bison se conoce como **árbol de derivación** o *parse tree*, y suele ser más detallado (y más cercano a la gramática) de lo estrictamente necesario para las fases posteriores.

3.5. Árbol de sintaxis abstracta

A partir del análisis sintáctico se puede construir un **árbol de sintaxis abstracta** (*Abstract Syntax Tree*, AST). A diferencia del árbol de derivación completo, el AST **elimina detalles sintácticos** que no son necesarios para las fases posteriores (como paréntesis, palabras clave redundantes o nodos intermedios de la gramática) y conserva principalmente la **estructura semánticamente relevante** del programa.

Para `int x = 10 + 20;`, un AST puede representarse como:

```
Declaration
|-- type: Int
|-- name: x
'-- value: BinaryExpression
    |-- left: IntegerLiteral(10)
    |-- operator: +
    '-- right: IntegerLiteral(20)
```

El AST permite que las fases siguientes (análisis semántico, generación de código intermedio) trabajen sobre una representación estructurada del programa, en lugar de procesar directamente los caracteres del código fuente o los detalles superfluos del árbol de derivación.

Además, la estructura del árbol permite representar propiedades sintácticas como la **precedencia de operadores**. Por ejemplo:

```
10 + 20 * 3
```

debe interpretarse como $10 + (20 \times 3)$ y no como $(10 + 20) \times 3$. El AST representa esta diferencia mediante la **posición de los nodos dentro del árbol**: el nodo `*` queda anidado como hijo del nodo `+`, y no al revés.

Relación con Bison. En una especificación de Bison, la construcción del AST se realiza típicamente mediante **acciones semánticas** escritas en C, asociadas a cada regla de la gramática (por ejemplo, con el pseudo-código `$$ = nuevoNodoBinario($1, PLUS, $3)`; dentro de la regla que reconoce $\langle \text{expresion} \rangle \rightarrow \langle \text{expresion} \rangle \text{ PLUS } \langle \text{termino} \rangle$). Esta técnica se conoce como **traducción dirigida por la sintaxis**, y es el mecanismo central que conecta la gramática formal con la construcción efectiva del AST en C.

3.6. Análisis semántico

El hecho de que un programa sea **sintácticamente correcto** no implica que sea **semánticamente válido**. El analizador semántico utiliza el árbol sintáctico (o el AST) y la información de la tabla de símbolos para comprobar restricciones que no pueden expresarse adecuadamente mediante una gramática libre de contexto —por ejemplo, que un identificador haya sido declarado antes de usarse, o que los tipos de una operación sean compatibles—.

Una parte central del análisis semántico es la **comprobación (verificación) de tipos**: el compilador verifica que cada operador tenga operandos del tipo esperado. Cuando el lenguaje lo permite, pueden aplicarse **coerciones**, es decir, conversiones automáticas de tipo (por ejemplo, de `int` a `float`); volveremos sobre esto con un ejemplo completo en la sección 4.

Por ejemplo:

```
int x = "hola";
```

puede tener una estructura **sintácticamente válida**, ya que respeta la forma general *tipo, identificador, asignación, expresión* exigida por la gramática, pero es **semánticamente incorrecta**, porque una cadena no puede asignarse a una variable de tipo `int`.

Para una declaración genérica `int x = e`; podemos establecer las siguientes reglas semánticas:

1. x debe ser un identificador válido.
2. x no debe haber sido declarado previamente en el mismo ámbito.
3. La expresión e debe producir un valor de tipo `int` (o un tipo que pueda coercionarse a `int`).
4. El valor producido por e se asigna a x .
5. x queda registrada en la tabla de símbolos como una variable de tipo `int`.

Para la expresión `10 + 20` se puede determinar:

$$10 : \text{int} \quad 20 : \text{int} \quad 10 + 20 : \text{int}$$

Por lo tanto, `int x = 10 + 20`; es semánticamente válida.

3.7. Tabla de símbolos

La **tabla de símbolos** es una estructura de datos utilizada por distintas fases del compilador para almacenar información sobre los nombres (identificadores) que aparecen en el programa: variables, funciones, tipos, etc. **No** constituye necesariamente una fase independiente, sino un **servicio transversal** utilizado sobre todo durante el análisis semántico y, posteriormente, durante la generación de código.

Entre los atributos que puede almacenar una entrada se encuentran:

- nombre;
- tipo;
- ámbito (*scope*);
- ubicación en memoria;
- para funciones/procedimientos: número y tipo de parámetros, forma de pasaje (por valor o por referencia) y tipo de retorno;

- información adicional necesaria para la generación de código.

La estructura de datos debe diseñarse para permitir **buscar, insertar y actualizar** cada entrada con rapidez (típicamente mediante una tabla hash o un árbol balanceado).

Para `int x = 10 + 20`; obtenemos:

Nombre	Tipo	Ámbito
<code>x</code>	<code>int</code>	local

Cuadro 2: Ejemplo de tabla de símbolos

Cuando el analizador léxico encuentra el lexema `x`, produce un token asociado a una entrada de esta tabla: `IDENTIFIER ->(id, 1)`. Las fases posteriores utilizan esta referencia para obtener toda la información disponible sobre el identificador, en lugar de volver a manipular directamente la cadena de caracteres `x`.

3.8. Manejo de errores

El compilador también debe **detectar y comunicar** los errores que encuentre durante el procesamiento del programa. Los errores pueden aparecer en cualquier fase:

- un carácter que no pertenece al lenguaje produce un **error léxico** (por ejemplo, un símbolo no reconocido por ningún patrón);
- una secuencia de tokens que no puede derivarse mediante la gramática produce un **error sintáctico**;
- una operación incompatible con los tipos involucrados produce un **error semántico**.

El manejo de errores es una actividad **transversal** a todo el proceso de compilación. El objetivo no es solamente detectar que existe un error, sino proporcionar información suficiente para que el programador pueda identificar su **ubicación** (por ejemplo, línea y columna) y su **naturaleza**. Tanto Flex como Bison ofrecen mecanismos básicos para esto: Flex puede reportar un carácter no reconocido por ningún patrón, y Bison invoca una rutina de manejo de error (habitualmente `yyerror`) cuando la secuencia de tokens no es derivable desde la gramática.

3.9. Generación de código intermedio

Después del análisis, el compilador puede construir una o más **representaciones intermedias** (*Intermediate Representation*, IR). Los árboles sintácticos (y el AST) son en sí mismos una forma de representación intermedia, utilizada durante el análisis sintáctico y semántico.

Muchos compiladores generan además una representación intermedia de **más bajo nivel**, similar a un lenguaje ensamblador abstracto para una máquina hipotética, que debe cumplir dos propiedades importantes: ser **fácil de producir** (a partir del AST) y **fácil de traducir** a la máquina destino. Una forma habitual de IR es el **código de tres direcciones**: una secuencia de instrucciones simples, con a lo sumo un operador y tres operandos por instrucción (cada operando puede actuar como un registro).

Para `int x = 10 + 20`;, una representación intermedia sencilla podría ser:

<pre>t1 = 10 + 20 x = t1</pre>

La representación intermedia permite **separar el lenguaje fuente del lenguaje/arquitectura destino**: en lugar de generar directamente código específico de una máquina, el compilador representa las operaciones del programa de forma independiente de la plataforma. Esto es justamente lo que aprovecharemos al usar **C como lenguaje destino** (sección 5): nuestro back-end sólo necesita traducir la IR a C, y delegamos en el compilador de C el trabajo de generar el código específico de cada arquitectura.

3.10. Optimización de código

Una vez construida la representación intermedia, pueden aplicarse transformaciones que **preserven el significado** del programa mientras mejoran alguna de sus propiedades: tiempo de ejecución, tamaño del código generado o consumo de energía, entre otras.

Por ejemplo:

```
t1 = 10 + 20
x = t1
```

puede transformarse en:

```
x = 30
```

Esta transformación, en la que una expresión constante se evalúa en tiempo de compilación, se conoce como **plegado de constantes** (*constant folding*). Las optimizaciones pueden aplicarse sobre expresiones, instrucciones, flujo de control o el uso de valores temporales, entre muchos otros aspectos.

Es fundamental notar que la optimización **no modifica el significado observable** del programa: su único objetivo es obtener una representación equivalente que se ejecute de manera más eficiente. Esta fase es **opcional** y su alcance varía mucho entre compiladores.

3.11. Generación de código destino

La última fase principal del compilador transforma la representación intermedia (optimizada o no) en el **lenguaje destino**. En un compilador tradicional, esto suele ser lenguaje ensamblador o código máquina de una arquitectura concreta; en nuestro caso, como se detalla en la sección 5, el lenguaje destino será **C**.

Conceptualmente, esta fase debe resolver cuestiones como:

- qué instrucciones (o construcciones del lenguaje destino) representan cada operación;
- dónde almacenar los valores (registros, memoria, variables temporales);
- qué registros utilizar (**asignación de registros**), cuando el destino es código máquina;
- cómo representar las estructuras de control;
- cómo organizar la memoria necesaria en tiempo de ejecución.

IR → selección de instrucciones → asignación de registros → código destino

3.12. Front-end y back-end

Las fases del compilador pueden agruparse conceptualmente en dos grandes bloques.

El **front-end** depende principalmente del **lenguaje fuente**. Sus responsabilidades incluyen:

- análisis léxico (sección 3.1);

- análisis sintáctico (sección 3.3);
- análisis semántico (sección 3.6);
- generación de la representación intermedia (sección 3.9).

El **back-end** recibe la representación intermedia y se ocupa de transformarla en código del lenguaje destino, incluyendo las optimizaciones y las decisiones relacionadas con la generación de código.

$$\text{Fuente} \rightarrow \boxed{\text{Front-end}} \rightarrow \text{IR} \rightarrow \boxed{\text{Back-end}} \rightarrow \text{Destino}$$

Esta separación tiene una ventaja práctica enorme: permite que **distintos lenguajes fuente compartan un mismo back-end**,

$$\begin{array}{lcl} \text{Lenguaje A} & \rightarrow & \text{Front-end A} \\ \text{Lenguaje B} & \rightarrow & \text{Front-end B} \end{array} \rightarrow \text{IR} \rightarrow \text{Back-end}$$

o que un mismo front-end se combine con **distintos back-ends**:

$$\text{Fuente} \rightarrow \text{Front-end} \rightarrow \text{IR} \rightarrow \begin{cases} \text{Back-end x86} \\ \text{Back-end ARM} \end{cases}$$

En nuestro proyecto de la materia, Flex y Bison nos ayudarán a construir precisamente el **front-end** (análisis léxico y sintáctico), mientras que nuestro back-end —más simple de lo habitual gracias a que el destino es C— se encargará de traducir la IR a código C válido.

3.13. Fases y pasadas

Es importante no confundir dos nociones distintas:

- Las **fases** representan las distintas **tareas lógicas** que debe realizar el compilador (léxico, sintáctico, semántico, generación de IR, optimización, generación de código).
- Una **pasada** es una recorrida completa sobre el programa (leyendo un archivo de entrada y escribiendo un archivo de salida) durante la cual se ejecutan una o más fases.

Una implementación concreta puede, por ejemplo, agrupar el análisis léxico, sintáctico, semántico y la generación de IR en una **primera pasada**:

$$\text{Fuente} \rightarrow \text{léxico} \rightarrow \text{sintáctico} \rightarrow \text{semántico} \rightarrow \text{IR}$$

realizar la optimización como una **segunda pasada** (opcional):

$$\text{IR} \rightarrow \text{IR optimizada}$$

y finalmente generar el código destino en una **tercera pasada**:

$$\text{IR optimizada} \rightarrow \text{código destino}$$

En resumen: las **fases** describen *qué* tareas conceptuales realiza el compilador; las **pasadas** describen *cómo* se organiza su implementación. En nuestro compilador, es habitual que Flex y Bison colaboren dentro de una misma pasada: Bison (el analizador sintáctico) va pidiendo tokens a Flex (el analizador léxico) **a demanda**, uno a la vez, en lugar de que Flex primero tokenice todo el archivo y luego Bison lo procese.

3.14. Herramientas para la construcción de compiladores

Las distintas fases de un compilador pueden implementarse manualmente o mediante herramientas especializadas que generan automáticamente componentes a partir de especificaciones de alto nivel. Entre las más comunes:

1. **Generadores de analizadores sintácticos (parsers)**, que producen automáticamente analizadores sintácticos a partir de una **descripción gramatical** de un lenguaje de programación. *Ejemplo: Bison.*
2. **Generadores de escáneres (analizadores léxicos)**, que producen analizadores léxicos a partir de una descripción de los tokens de un lenguaje mediante **expresiones regulares**. *Ejemplo: Flex.*
3. **Motores de traducción orientados a la sintaxis**, que producen rutinas para recorrer el árbol sintáctico y generar código intermedio (esto es, en esencia, lo que hacemos al escribir acciones semánticas en las reglas de Bison).
4. **Generadores de generadores de código**, que producen un generador de código a partir de reglas para traducir cada operación del lenguaje intermedio al lenguaje/arquitectura destino.
5. **Motores de análisis de flujo de datos**, utilizados para obtener información sobre cómo se propagan los valores por el programa; son clave para muchas optimizaciones.
6. **Kits de construcción de compiladores**, que proporcionan un conjunto integrado de componentes para implementar varias de estas tareas a la vez.

Estas herramientas **no constituyen fases adicionales**: son mecanismos que facilitan la implementación de las fases que ya existen conceptualmente. Flex y Bison son, respectivamente, un generador de escáneres y un generador de parsers: no añaden ninguna fase nueva al esquema de la figura de la sección 3.15, sólo **automatizan** la construcción de las dos primeras.

3.15. Proceso completo

El proceso completo puede resumirse de la siguiente manera:

Fuente → Análisis léxico → Tokens

Tokens → Análisis sintáctico → AST

AST → Análisis semántico → AST validado

AST validado → Código intermedio → Optimización

IR optimizada → Generación de código → Código destino

En cada una de estas fases intervienen estructuras auxiliares transversales, como la **tabla de símbolos** (sección 3.7) y los mecanismos de **manejo de errores** (sección 3.8).

4. Ejemplo completo: recorrido de una instrucción a través de todas las fases

Para consolidar la relación entre todas las fases, es útil seguir un segundo ejemplo —clásico del Libro del Dragón— algo más rico que `int x = 10 + 20;`, porque permite ver en acción la **coerción de tipos** y una **optimización real**. Consideremos la instrucción de asignación:

```
posicion = inicial + velocidad * 60
```

donde `posicion`, `inicial` y `velocidad` son variables de tipo `float`.

1. Análisis léxico. Los caracteres se agrupan en lexemas y se traducen a tokens:

$\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$

donde la entrada 1 de la tabla de símbolos corresponde a `posicion`, la 2 a `inicial` y la 3 a `velocidad`. El lexema 60 da lugar, en rigor, a un token con su propia entrada en la tabla de símbolos para representar el entero.

2. Análisis sintáctico. A partir de esos tokens se construye un árbol sintáctico que refleja la precedencia usual de los operadores (la multiplicación se agrupa antes que la suma):

```
=
|-- <id,1>          (posicion)
'-- +
    |-- <id,2>       (inicial)
    '-- *
        |-- <id,3>    (velocidad)
        '-- 60
```

El nodo raíz `=` indica que el resultado de la suma se almacena en la ubicación de `posicion`; el nodo `+` indica que primero debe sumarse `inicial` al resultado de la multiplicación; el nodo `*` indica que `velocidad` se multiplica por 60 **antes** de sumarse, en concordancia con las reglas usuales de precedencia aritmética.

3. Análisis semántico. Supongamos que `posicion`, `inicial` y `velocidad` están declaradas como `float`, mientras que el lexema 60 es, por sí solo, un entero. El comprobador de tipos detecta que el operador `*` se aplica a un `float` (`velocidad`) y a un `int` (60). Como el lenguaje permite la **coerción** de `int` a `float` en operaciones aritméticas mixtas, el analizador semántico inserta de manera explícita un operador de conversión `inttofloat` sobre el 60. El árbol resultante incorpora entonces ese nodo adicional antes de pasar a la siguiente fase.

4. Generación de código intermedio. Usando código de tres direcciones, con una instrucción por cada operador del árbol (incluyendo la conversión de tipos), se obtiene:

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Aquí `id1`, `id2` e `id3` son las referencias a las entradas de la tabla de símbolos de `posicion`, `inicial` y `velocidad`, respectivamente, y `t1`, `t2`, `t3` son nombres temporales generados por el compilador. Obsérvese que cada instrucción de tres direcciones tiene, como máximo, un operador del lado derecho, lo cual fuerza a explicitar el orden en el que se realizan las operaciones (la multiplicación queda calculada antes que la suma, tal como exigía el árbol sintáctico).

5. Optimización de código. El optimizador puede advertir que la conversión de 60 a `float` puede resolverse en tiempo de compilación, reemplazando el entero 60 por la constante 60.0; también puede notar que `t3` se usa una única vez, así que su valor puede propagarse directamente a `id1`. El resultado es una secuencia más corta:

```
t1 = id3 * 60.0
id1 = id2 + t1
```

6. Generación de código destino. Si el destino fuera código máquina, y se usaran los registros R1 y R2, podría generarse:

```
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```

En nuestro proyecto, en cambio, el back-end no genera ensamblador ni código máquina: traduce la IR optimizada a **código C**, delegando la selección de instrucciones y la asignación de registros al compilador de C. Para la secuencia anterior, el back-end podría emitir algo equivalente a:

```
float t1;
t1 = velocidad * 60.0;
posicion = inicial + t1;
```

Este recorrido completo —léxico, sintáctico, semántico (con coerción), IR, optimización y generación de código— es el mismo esquema que aplicaremos, en miniatura, a `int x = 10 + 20`; durante la práctica con Flex y Bison.

5. Uso de C como lenguaje destino

En nuestro compilador se utilizará **C** como lenguaje destino. Esto significa que el compilador **no generará** directamente código máquina ni ensamblador: en su lugar, generará un programa escrito en C que representa las operaciones del lenguaje fuente. El proceso completo será:

Lenguaje fuente → Análisis léxico → Análisis sintáctico → Análisis semántico

↓

IR → Código C → Compilador de C → Código máquina

El objetivo de nuestro back-end será, por lo tanto, traducir la representación intermedia a código C válido. Por ejemplo, si la IR contiene:

```
t1 = 10 + 20
x = t1
```

el back-end podría generar:

```
int x;
x = 10 + 20;
```

y será el compilador de C ya existente quien se encargue de las etapas necesarias para producir código específico de la plataforma final.

Esta decisión simplifica considerablemente la construcción del back-end: los compiladores de C disponibles ofrecen optimizaciones maduras y soporte para múltiples arquitecturas, por lo que

nuestro proyecto no necesita reimplementar instrucciones, registros ni convenciones específicas de cada arquitectura (lo que sí sería necesario si generáramos código máquina directamente). La utilización de C como lenguaje destino permite concentrar el esfuerzo del proyecto en el **diseño y procesamiento de nuestro propio lenguaje** —justamente, el front-end que construiremos con Flex y Bison— delegando en el compilador de C la generación final de código para la plataforma correspondiente.

6. De la teoría del Capítulo 1 a la práctica: bases para Flex y Bison

Esta sección conecta explícitamente los conceptos ya vistos con las herramientas que comenzaremos a usar mañana.

6.1. Expresiones regulares como patrones léxicos: la base de Flex

En la sección 3.2 definimos un **patrón** como la regla que describe qué cadenas de caracteres pueden formar los lexemas de un token dado. El modelo formal estándar para expresar esas reglas de manera precisa y compacta son las **expresiones regulares**, apoyadas en el modelo de las **máquinas de estados finitos** (autómatas finitos). Estos son, junto con las gramáticas libres de contexto, de los modelos matemáticos más básicos usados en el diseño de compiladores.

Flex es un **generador de analizadores léxicos**: recibe un archivo de especificación (típicamente `.l` o `.flex`) formado por una lista de pares (*expresión regular*, *acción en C*), y genera automáticamente el código C de una función (`yylex()`) que:

1. lee el flujo de caracteres del programa fuente;
2. en cada paso, encuentra el **lexema más largo** que coincide con alguno de los patrones especificados (la llamada regla de la *coincidencia más larga*);
3. ejecuta la acción en C asociada, que típicamente construye y devuelve el **token** correspondiente —exactamente el par $\langle \text{nombre-token}, \text{valor-atributo} \rangle$ descrito en la sección 3.1—;
4. ignora (o trata especialmente) los espacios en blanco, tal como hace cualquier analizador léxico.

En otras palabras: **todo lo que hicimos “a mano” en la sección 3.1 (agrupar caracteres en lexemas, asignarles una categoría) es exactamente lo que Flex automatiza**, a partir de una tabla de patrones como la de la tabla 1.

6.2. Gramáticas libres de contexto y árboles sintácticos: la base de Bison

En la sección 3.4 vimos que la sintaxis de un lenguaje se especifica mediante una **gramática libre de contexto**: un conjunto de producciones que indican cómo se combinan los tokens (símbolos terminales) y las categorías sintácticas (símbolos no terminales) para formar construcciones válidas.

Bison es un **generador de analizadores sintácticos**: recibe un archivo de especificación (típicamente `.y` o `.yacc`) formado por una gramática —muy similar en notación a la de la sección 3.4— junto con **acciones semánticas** en C asociadas a cada producción, y genera automáticamente el código C de una función (`yyparse()`) capaz de:

1. pedir tokens, uno a la vez, al analizador léxico generado por Flex (`yylex()`);
2. determinar si la secuencia de tokens recibida es derivable desde el símbolo inicial de la gramática, usando un algoritmo de análisis ascendente de tipo **LALR**;

3. ejecutar, en cada reducción de una producción, la acción semántica en C asociada —lo que en la sección 3.5 llamamos **traducción dirigida por la sintaxis**—, típicamente para construir un nodo del **árbol de sintaxis abstracta**;
4. reportar un error sintáctico (invocando `yyerror`) cuando ninguna producción de la gramática permite continuar el análisis.

Es decir: **todo lo que hicimos “a mano” en las secciones 3.3 y 3.5 (construir el árbol sintáctico y luego el AST a partir de la gramática de la sección 3.4) es exactamente lo que Bison automatiza.**

6.3. El flujo Flex + Bison y su correspondencia con las fases del compilador

En una implementación típica con estas herramientas, Flex y Bison **no se ejecutan como dos pasadas separadas**: Bison controla el proceso y pide tokens a Flex **bajo demanda**, uno por vez, tal como se explicó en la sección 3.13. La correspondencia con las fases del capítulo es directa:

Fase del compilador	Herramienta	Entrada / salida
Análisis léxico	Flex (<code>yylex</code>)	caracteres → tokens
Análisis sintáctico	Bison (<code>yyparse</code>)	tokens → árbol / AST
Traducción dirigida por la sintaxis	acciones en Bison	AST anotado
Análisis semántico	código C del programador	AST validado

Cuadro 3: Correspondencia entre las fases del Capítulo 1 y el par Flex/Bison

En síntesis: **Flex y Bison automatizan el front-end** del compilador (secciones 3.1 a 3.9), pero **no reemplazan** el diseño conceptual que vimos en este capítulo: antes de escribir una sola línea de `.l` o `.y`, necesitamos tener claros los **patrones léxicos** (expresiones regulares) y la **gramática** de nuestro lenguaje, exactamente como se plantearon en las tablas y producciones de las secciones 3.2 y 3.4. Ese es, precisamente, el contenido que repasaremos en la clase de mañana antes de comenzar con la implementación.

7. Panorama complementario

7.1. La evolución de los lenguajes de programación

Las primeras computadoras (década de 1940) se programaban directamente en **lenguaje máquina**. El desarrollo de los **lenguajes ensambladores**, a inicios de 1950, introdujo mne-mónicos y, más adelante, macroinstrucciones. En la segunda mitad de esa década surgieron los primeros **lenguajes de alto nivel** (Fortran, Cobol, Lisp), pensados para que la programación fuera más natural en distintos dominios.

Una clasificación habitual es por **generación**: 1ª generación (lenguaje máquina), 2ª (ensam-bladores), 3ª (lenguajes de alto nivel de propósito general, como C, C++, Java), 4ª (lenguajes de aplicación específica, como SQL) y 5ª (lenguajes basados en lógica y restricciones, como Prolog). Otra clasificación distingue lenguajes **imperativos** (especifican *cómo* realizar un cálculo, con una noción explícita de estado, como C) de los **declarativos** (especifican *qué* calcular, como los lenguajes funcionales o de lógica).

Cada avance en los lenguajes de programación impuso nuevas exigencias sobre quienes escri-ben compiladores, que debieron idear algoritmos y representaciones para dar soporte a las nuevas características, aprovechando además la evolución de la arquitectura de las máquinas.

7.2. La ciencia de construir un compilador

El diseño de compiladores combina, de forma ejemplar, **teoría matemática** y **validación empírica**: se formula una abstracción matemática que capture las características clave de un problema (por ejemplo, autómatas finitos y expresiones regulares para el léxico, gramáticas libres de contexto para la sintaxis, árboles para representar programas) y se resuelve con técnicas matemáticas, refinando la solución con experimentación.

Toda optimización de código debe cumplir ciertos objetivos de diseño, en orden de importancia:

1. Debe ser **correcta**: preservar el significado del programa compilado. Este es, con diferencia, el objetivo más importante: un compilador que genere código incorrecto no sirve, sin importar qué tan rápido sea el código resultante.
2. Debe **mejorar el rendimiento** de una amplia variedad de programas de entrada.
3. El **tiempo de compilación** debe mantenerse razonable, para no entorpecer el ciclo de desarrollo y depuración.
4. El **esfuerzo de ingeniería** requerido debe ser manejable.

Vale la pena notar que muchos de los problemas centrales de la optimización de compiladores son **indecidibles** en general; por eso buena parte del trabajo consiste en formular heurísticas razonables, más que en buscar soluciones perfectas.

7.3. Aplicaciones de la tecnología de compiladores

Las técnicas desarrolladas para construir compiladores se aplican mucho más allá de la compilación de lenguajes de programación de propósito general. Algunos ejemplos mencionados en el capítulo:

- **Traducción binaria**: convertir código máquina de una arquitectura a otra.
- **Síntesis de hardware**: traducir descripciones en lenguajes como VHDL o Verilog a circuitos concretos.
- **Interpretes de consultas de bases de datos**: compilar o interpretar lenguajes como SQL.
- **Simulación compilada**: compilar un diseño para simularlo mucho más rápido que interpretándolo.
- **Herramientas de productividad de software**: comprobación de tipos, análisis de flujo de datos para detectar errores (referencias nulas, desbordamientos de buffer) y herramientas de administración de memoria (por ejemplo, recolección de basura), todas ellas apoyadas en técnicas de análisis de programas nacidas en el contexto de la optimización de compiladores.

8. Conclusión

A partir del modelo presentado en este capítulo, el desarrollo de nuestro compilador puede organizarse como una secuencia de etapas claramente diferenciadas.

En primer lugar, debemos definir la **especificación léxica** del lenguaje: sus tokens, lexemas y patrones (expresiones regulares), que es exactamente lo que especificaremos en el archivo de Flex. Luego debemos establecer la **gramática** que determina cómo se combinan esos tokens, que es lo que especificaremos en el archivo de Bison, junto con las acciones semánticas que construyen

el AST. Sobre esa estructura incorporaremos las **reglas semánticas** y la **tabla de símbolos** necesarias para validar los programas. Una vez definido el front-end, estableceremos una **representación intermedia** que exprese las operaciones del lenguaje de manera independiente del código C generado, y finalmente implementaremos el **back-end** encargado de traducir esa representación a C.

De esta manera, el desarrollo seguirá la organización clásica de un compilador:

$$\text{Lenguaje} \rightarrow \text{Léxico} \rightarrow \text{Sintaxis} \rightarrow \text{Semántica} \rightarrow \text{IR} \rightarrow \text{C}$$

y, en la práctica concreta de la cátedra, las dos primeras flechas de ese esquema —**léxico** y **sintaxis**— son precisamente el territorio de **Flex** y **Bison**, las herramientas que comenzaremos a estudiar en la próxima clase.